universität ulm

Numerical Optimization
Winter semester 2025/26

Exercise sheet 5 for Numerical Optimization

Manuel Caipo
Degree program: M.Sc. Computational Science Engineering
Enrollment number: 30783212

Instructors: Dr. Michael Lehn, Tobias Born

19. November 2025 - 89081 Ulm

Chapter 1

Solutions Exercise sheet 4

1.1 Exercise 1

1.1.1 Part a)

Script were read.

1.1.2 Part b)

```
1
2
3  function[] = nelder_mead(f, S, tol, kmax, alpha, beta, gamma, sigma, p_plot, t_plot)
4
5  n = size(S,2)-1;
6  k = 0;
7  curr_ops = {};
8
9  % evaluate f at vertices
10 F = zeros(1,n+1);
11 for j=1:n+1
12     F(j) = f(S(:,j));
13 end
14
15 % sort vertices in ascending order of function value
16 [F,idx] = sort(F);
17 S = S(:,idx);
18
19 % compute barycenter of best n vertices
20 xc = mean(S(:,1:n),2);
21 fc = f(xc);
22
23 % termination criterion
24 while ( (1/(n+1))*sum( (F - f(xc)).^2 ) > tol ) && (k < kmax)
25
26     % compute xr (reflection point)
27     xr = xc + alpha*(xc - S(:,n+1));
28     fr = f(xr);
29
30     % first case: f1 <= fr <= fn
31     if (F(1) <= fr) && (fr <= F(n))
32
33         S(:,n+1) = xr;
34         F(n+1)   = fr;
35         curr_ops{end+1} = 'REFLECTION';
36
37     % second case: fr < f1
38     elseif fr < F(1)
```

```
39
40          % compute xe (extrapolation point)
41          xe = xc + beta*(xr - xc);
42          fe = f(xe);
43
44          % if extrapolation point better than reflection point
45          if fe < fr
46              S(:,n+1) = xe;
47              F(n+1) = fe;
48              curr_ops{end+1} = 'EXPANSION';
49          else
50              % new vertex = reflection point
51              S(:,n+1) = xr;
52              F(n+1) = fr;
53              curr_ops{end+1} = 'REFLECTION';
54          end
55
56      % third case: fr > fn
57      elseif fr > F(n)
58
59          % if fr not smaller than fn+1  (fr >= F(n+1))
60          if fr >= F(n+1)
61
62              % compute xk (contraction) wrt xn+1
63              xk = xc + gamma*(S(:,n+1) - xc);
64              fk = f(xk);
65
66              % if fk smaller than fn+1
67              if fk < F(n+1)
68                  S(:,n+1) = xk;
69                  F(n+1) = fk;
70                  curr_ops{end+1} = 'CONTRACTION';
71
72              % if fk not smaller than fn+1 -> SHRINK
73              else
74                  for j = 2:n+1
75                      S(:,j) = S(:,1) + sigma*(S(:,j) - S(:,1));
76                      F(j) = f(S(:,j));
77                  end
78                  curr_ops{end+1} = 'SHRINKING';
79              end
80
81          % if fr < fn+1
82          elseif fr < F(n+1)
83
84              % contraction wrt xr
85              xk = xc + gamma*(xr - xc);
86              fk = f(xk);
87
88              if fk < fr
89                  S(:,n+1) = xk;
90                  F(n+1) = fk;
91                  curr_ops{end+1} = 'CONTRACTION';
92
93              else
94                  % shrink
95                  for j = 2:n+1
96                      S(:,j) = S(:,1) + sigma*(S(:,j) - S(:,1));
97                      F(j) = f(S(:,j));
98                  end
99                  curr_ops{end+1} = 'SHRINKING';
100             end
101
```

```
        end
    end

    % sort vertices again
    [F,idx] = sort(F);
    S = S(:,idx);

    % compute barycenter again
    xc = mean(S(:,1:n),2);

    % increase iteration step
    k = k + 1;

    % plotting (ignore)
    pause(1);
    set(p_plot, 'XData', [S(1,:),S(1,1)], 'YData', [S(2,:),S(2,1)]);
    if numel(curr_ops) > 4
        curr_ops(1) = [];
    end
    tmp = curr_ops{end};
    curr_ops{end} = ['\bf\underline{' curr_ops{end} '}'];
    set(t_plot, 'String', strjoin(curr_ops,', '));
    curr_ops{end} = tmp;

end

end

\subsection{Part c-d)}
```

Nelder Mead

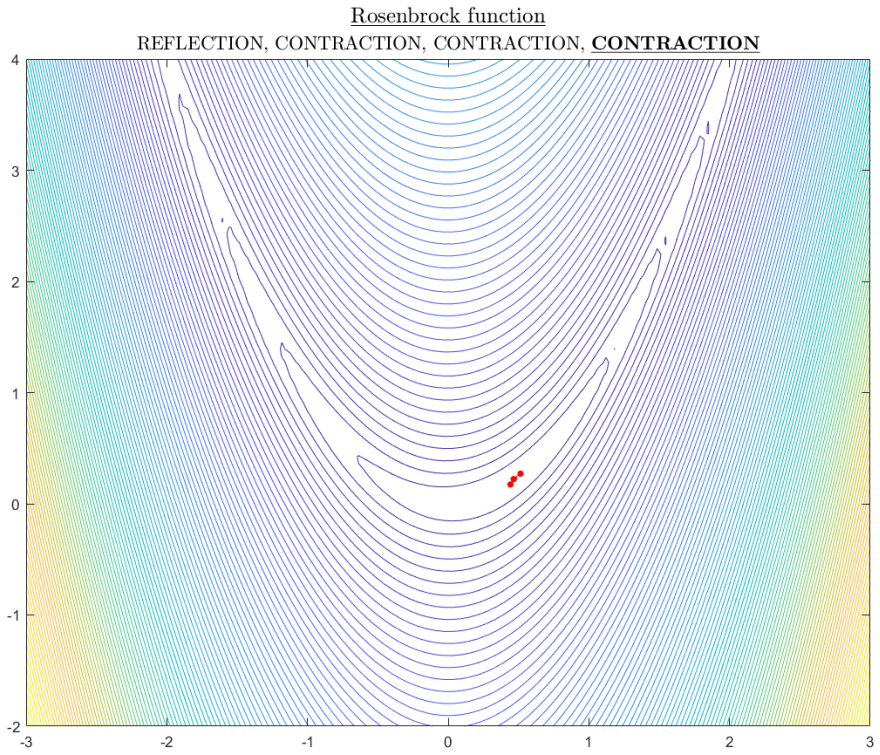


Figure 1.1: Convergence in rosenbrock

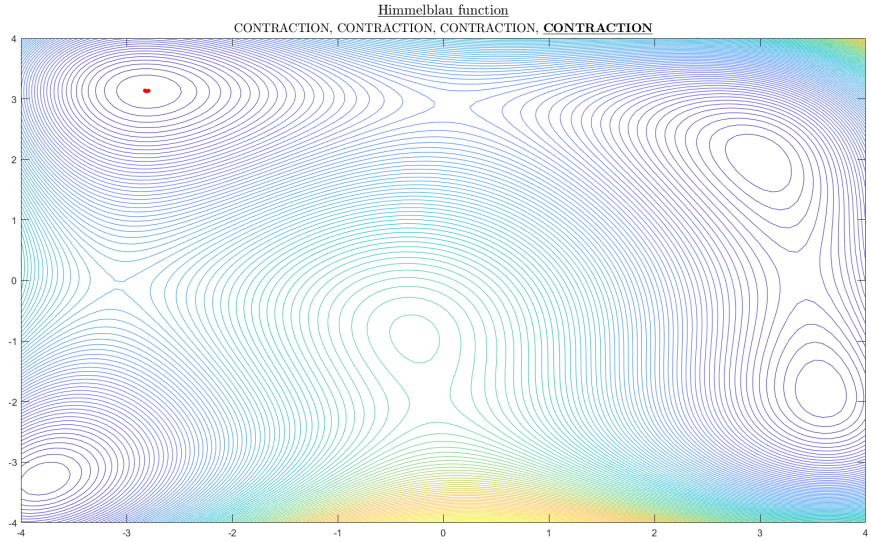


Figure 1.2: Convergence in himmelblau

Rosenbrock Function

- **Characteristics:** Single global minimum within a narrow, curved valley.
- **Behavior:** The Nelder-Mead method often fails to enter this valley correctly and progresses very slowly.
- **Result:** The maximum iteration limit is typically reached before the tolerance is met, leading to **non-convergence**.
- **Conclusion:** Nelder-Mead struggles with ill-conditioned functions like Rosenbrock.

Himmelblau Function

- **Characteristics:** Four distinct global minima.
- **Behavior:** Depending on the initial simplex, the method converges to different minima.
- **Result:** The tolerance criterion is almost always met; thus, the method **converges**.
- **Conclusion:** The various outcomes reflect the multi-modal nature of the problem and the local search behavior of Nelder-Mead.

1.2 Exercise 2

1.2.1 Part a)

Descent Directions and Step Sizes let $x_k \in \mathbb{R}^n$ be fixed, let $f : \mathbb{R}^n \to \mathbb{R}$ be continuously differentiable, and let $d_k \in \mathbb{R}^n$ satisfy $\|d_k\|_2 = 1$.
Assume that the direction d_k satisfies

$$\nabla f(x_k)^T d_k < 0. \tag{1.1}$$

We must prove that there exists a number $\tilde{\alpha} > 0$ such that

$$f(x_k + \alpha d_k) < f(x_k) \qquad \text{for all } \alpha \in (0, \tilde{\alpha}). \tag{1.2}$$

Reduction to a one-dimensional function. Define the scalar function

$$g(t) := f(x_k + t\, d_k), \qquad t \in \mathbb{R}. \tag{1.3}$$

Notice that

$$g(0) = f(x_k), \qquad g(\alpha) = f(x_k + \alpha d_k). \tag{1.4}$$

Thus the desired inequality

$$f(x_k + \alpha d_k) < f(x_k)$$

is equivalent to

$$g(\alpha) < g(0).$$

Derivative of g. Since f is continuously differentiable, the chain rule yields

$$g'(t) = \nabla f(x_k + t d_k)^T d_k. \tag{1.5}$$

Evaluating at $t = 0$ gives

$$g'(0) = \nabla f(x_k)^T d_k < 0. \tag{1.6}$$

One-dimensional consequence. From elementary one-dimensional calculus: If a differentiable function $h : \mathbb{R} \to \mathbb{R}$ satisfies $h'(0) < 0$, then there exists $\delta > 0$ such that

$$h(t) < h(0) \qquad \text{for all } t \in (0, \delta). \tag{1.7}$$

This follows directly from the definition of the derivative at 0,

$$h'(0) = \lim_{t \to 0} \frac{h(t) - h(0)}{t},$$

because a negative derivative means that the quotient $\frac{h(t)-h(0)}{t}$ must be negative for sufficiently small positive t.
Applying this to g, since $g'(0) < 0$, there exists $\tilde{\alpha} > 0$ such that

$$g(\alpha) < g(0) \qquad \forall\, \alpha \in (0, \tilde{\alpha}). \tag{1.8}$$

Returning to f,

$$f(x_k + \alpha d_k) = g(\alpha) < g(0) = f(x_k). \tag{1.9}$$

Thus the claim is proved.

1.2.2 Part b)

(Steepest descent direction and its uniqueness The steepest descent direction is defined by

$$d_k = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|_2}. \tag{1.10}$$

We must show that this direction satisfies

$$\nabla f(x_k)^T d_k = \min_{\|d\|_2=1} \nabla f(x_k)^T d, \tag{1.11}$$

and that it is the unique minimizer.

Step 1: Cauchy–Schwarz inequality. For any vectors $a, b \in \mathbb{R}^n$, the Cauchy–Schwarz inequality states:

$$|a^T b| \leq \|a\|_2 \, \|b\|_2. \tag{1.12}$$

Applying this with $a = \nabla f(x_k)$ and any d with $\|d\|_2 = 1$, we obtain:

$$|\nabla f(x_k)^T d| \leq \|\nabla f(x_k)\|_2. \tag{1.13}$$

Hence,

$$\nabla f(x_k)^T d \geq -\|\nabla f(x_k)\|_2 \qquad \text{for all } \|d\|_2 = 1. \tag{1.14}$$

This shows that no unit vector can yield a value lower than $-\|\nabla f(x_k)\|_2$.

Step 2: The steepest descent direction achieves the lower bound. Let

$$d_k = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|_2}.$$

Its norm is

$$\|d_k\|_2 = \frac{\|\nabla f(x_k)\|_2}{\|\nabla f(x_k)\|_2} = 1. \tag{1.15}$$

Then

$$\begin{aligned} \nabla f(x_k)^T d_k &= \nabla f(x_k)^T \left(-\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|_2} \right) \\ &= -\frac{\|\nabla f(x_k)\|_2^2}{\|\nabla f(x_k)\|_2} \\ &= -\|\nabla f(x_k)\|_2. \end{aligned} \tag{1.16}$$

This exactly equals the lower bound obtained above. Thus,

$$\nabla f(x_k)^T d_k = \min_{\|d\|_2=1} \nabla f(x_k)^T d. \tag{1.17}$$

Step 3: Uniqueness. Equality in Cauchy–Schwarz holds if and only if the vectors are linearly dependent. Thus, for equality to hold in

$$\nabla f(x_k)^T d \geq -\|\nabla f(x_k)\|_2,$$

the vector d must satisfy

$$d = \lambda \nabla f(x_k) \qquad \text{for some scalar } \lambda. \tag{1.18}$$

Since $\|d\|_2 = 1$, we obtain

$$|\lambda| = \frac{1}{\|\nabla f(x_k)\|_2}. \tag{1.19}$$

To obtain the negative value $-\|\nabla f(x_k)\|_2$, we require $\lambda < 0$, hence

$$\lambda = -\frac{1}{\|\nabla f(x_k)\|_2}. \tag{1.20}$$

Thus the only possible minimizer is

$$d = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|_2} = d_k. \tag{1.21}$$

Therefore the steepest descent direction is the unique minimizer.

1. The steepest descent direction achieves the smallest possible value of $\nabla f(x_k)^T d$ among all unit vectors.
2. This direction is the only unit vector that achieves this minimum.

1.2.3 Part c - d)

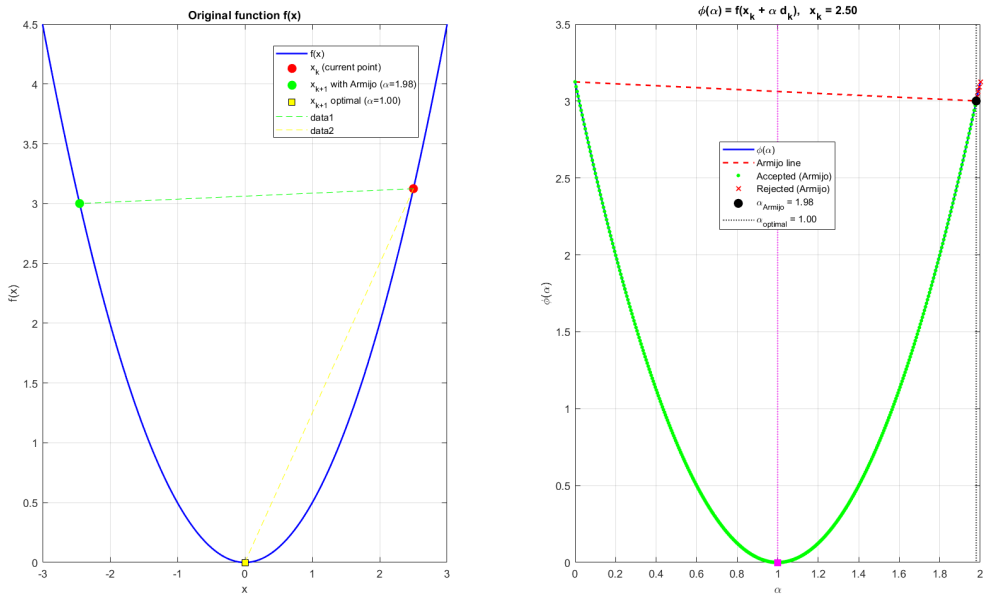


Figure 1.3: Armijo case

c_1 **(Armijo condition)**

- prevents step sizes that are too large,
- requires a sufficient decrease in the function value,
- controls the height of the Armijo (red) line.

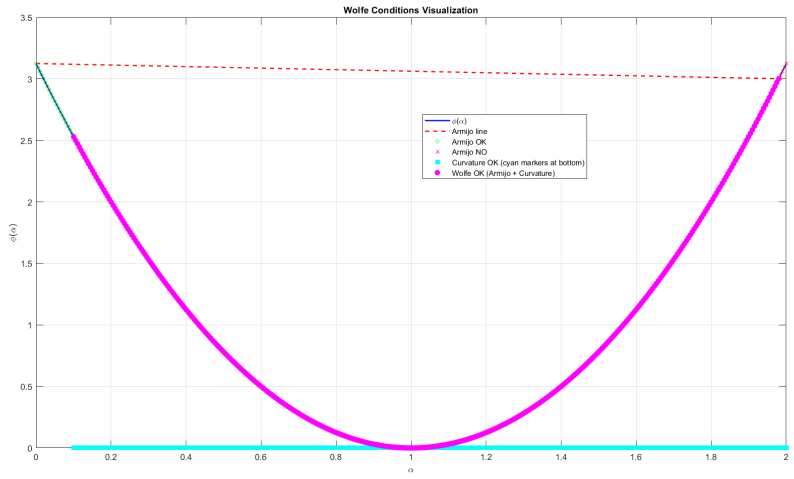


Figure 1.4: Curvature case

c_2 (curvature condition)

- prevents step sizes that are too small,
- requires that the slope (directional derivative) changes after the step,
- controls the final slope after the step.

Wolfe conditions = Armijo + Curvature

- guarantee a step size that is neither too large nor too small,
- commonly used in methods such as BFGS, Newton, and Conjugate Gradient.

1.2.4 Part e)

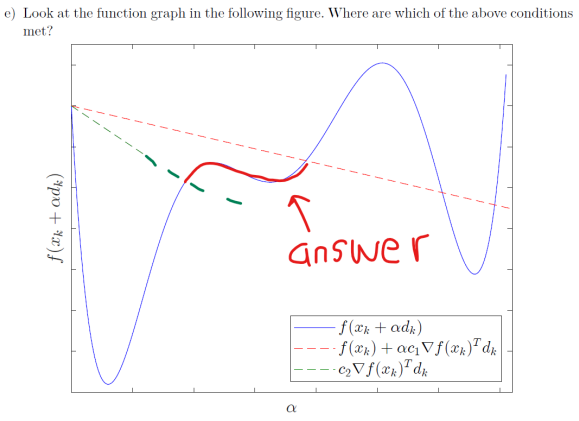


Figure 1.5: Curvature case

From the plot we can see:
- The *Armijo condition* is satisfied wherever $f(x_k + \alpha d_k)$ (blue curve) lies below the red dashed line. This happens in intervals around the local minima of the blue curve.
- The *curvature condition* is satisfied in regions where the slope of $f(x_k + \alpha d_k)$ has become sufficiently flat, i.e. near the local minima, so that the directional derivative is above the green dashed line.

- The *Wolfe conditions* (Armijo + curvature) are met only on the intersection of these two regions, i.e. on a smaller interval around the local minimum where the blue curve is below the red line and the slope has already changed enough. This corresponds roughly to the middle segment around the small local minimum.

1.2.5 Part f)

For a onedimensional function we set $\phi(\alpha) = f(x_k + \alpha d_k)$. Then condition (6) becomes

$$|\phi'(\alpha)| \leq c_2 |\phi'(0)|.$$

This means that the slope of f along the search direction after taking the step α is much smaller in magnitude than the initial slope. In other words, we have moved to a point where the function is almost flat in the search direction, i.e. close to a stationary point.

1.2.6 Part g)

In the figure from exercise (e) the strong Wolfe conditions are met only on a small interval around the local minimum of $f(x_k + \alpha d_k)$. On this interval

- the blue curve lies below the red Armijo line (Armijo condition), and
- the slope of the blue curve is almost zero, so that $|\nabla f(x_k + \alpha d_k)^T d_k|$ is much smaller than $|\nabla f(x_k)^T d_k|$ (strong curvature condition).

Outside this neighbourhood the function either does not decrease sufficiently (Armijo fails) or the slope is too large in magnitude (strong curvature fails.

1.3 Exercise 3

1.3.1 Part a)

```
1
2  function X = descent_method(x0, grad_f, direction, step_size, max_it, tol)
3  % DESCENT_METHOD  General implementation of a descent method
4  %
5  %   X = descent_method(x0, grad_f, direction, step_size, max_it, tol)
6  %
7  % INPUT:
8  %   x0         : initial point (column vector, usually dimension 2)
9  %   grad_f     : function handle for the gradient, grad_f(x)
10 %   direction  : function handle for the search direction, direction(x)
11 %   step_size  : function handle for the step size, step_size(x,d)
12 %   max_it     : maximum number of iterations
13 %   tol        : tolerance for ||grad_f(x)|| (stopping condition)
14 %
15 % OUTPUT:
16 %   X          : matrix where each column is an iterate x_k
17
18     % initialize with starting point
19     x = x0;
20     X = x0;   % store the first point
21
22     for k = 1:max_it
23         g = grad_f(x);          % compute gradient at the current point
24
25         % stopping condition: gradient is small enough
26         if norm(g) < tol
27             break;
28         end
29
30         % 1) compute the search direction d_k
31         d = direction(x);
32
33         % 2) compute the step size alpha_k
34         alpha = step_size(x, d);
35
36         % 3) update the point
37         x = x + alpha * d;
38
39         % store the new iterate
40         X(:, end+1) = x;
41     end
42 end
```

Descent Method

1.3.2 Part b)

```
1
2  function alpha = armijo_step_size(x, d, f, grad_f, beta, sigma)
3  % ARMIJO_STEP_SIZE  Armijo rule (backtracking line search)
4  %
5  %   alpha = armijo_step_size(x, d, f, grad_f, beta, sigma)
6  %
7  % INPUT:
8  %   x      : current point
9  %   d      : search direction
10 %   f      : function handle of the objective function, f(x)
11 %   grad_f : function handle for the gradient, grad_f(x)
```

```
%   beta   : reduction factor (0 < beta < 1), for example 0.5
%   sigma  : Armijo parameter (0 < sigma < 1), for example 1e-4
%
% OUTPUT:
%   alpha  : step size that satisfies the Armijo condition

    alpha = 1;                      % initial step size
    fx = f(x);                      % current function value
    gx = grad_f(x);                 % current gradient
    dir_deriv = gx' * d;            % directional derivative along d

    % Armijo condition:
    % f(x + alpha*d) <= f(x) + sigma * alpha * grad_f(x)' * d
    %
    % while this condition is NOT satisfied, we keep shrinking alpha

    while f(x + alpha * d) > fx + sigma * alpha * dir_deriv
        alpha = beta * alpha;       % backtracking: alpha, beta*alpha, beta^2*alpha,
    ...

        %     break;
        % end
    end
end
```

Armijo step size

1.3.3 Part c)

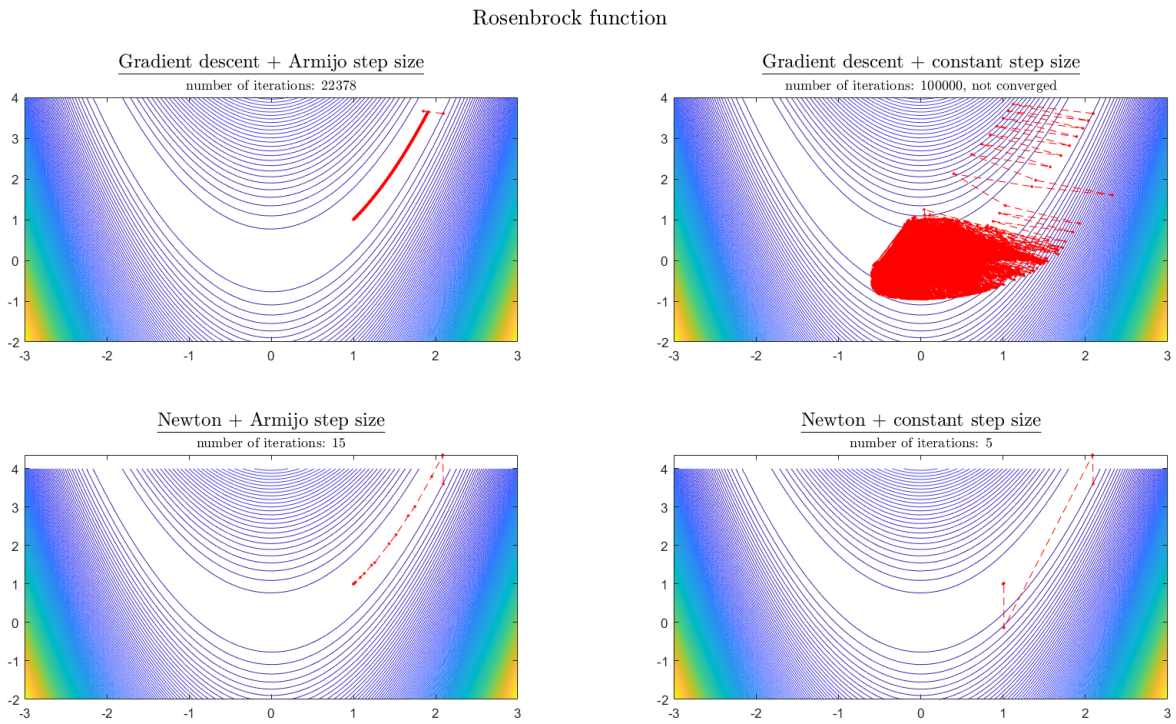


Figure 1.6: Rosenbrock Comparison

Rosenbrock function:

$$f(x, y) = (1 - x)^2 + 100\,(y - x^2)^2,$$

a classical example of a *narrow, curved, and badly conditioned* valley with minimum at $(1, 1)$. It is specifically designed to expose weaknesses of optimization algorithms: the gradient often points outside the valley, the curvature varies drastically, and the directions of steepest descent are poorly aligned with the path toward the minimizer.

Gradient Descent with Armijo Step Size The method converges but is extremely slow (22,378 iterations).

Mathematical interpretation. In the Rosenbrock valley, the gradient is almost flat along the valley but very steep across it. Thus, the gradient direction points largely *perpendicularly* to the correct path and sticks to the valley wall. Armijo line search avoids excessively large steps but reduces the step size so aggressively that progress becomes painfully slow.
Gradient descent technically works, but it is highly inefficient for narrow and curved valleys.

Gradient Descent with Constant Step Size $\alpha = 1$ This method fails to converge (100,000 iterations, chaotic behaviour).

Mathematical interpretation. Because the gradient magnitude varies dramatically across the domain, a fixed step size is inappropriate. In regions of high curvature, the step $\alpha = 1$ is far too large: the iterate repeatedly overshoots the valley and oscillates or diverges.
Fixed step sizes are unreliable for badly conditioned functions; step sizes must adapt to local curvature.

Newton's Method with Armijo Step Size This method converges rapidly (15 iterations).

Mathematical interpretation. Newton's direction

$$d_k = -H(x_k)^{-1} \nabla f(x_k)$$

incorporates second-order curvature information. For Rosenbrock, this is crucial: Newton corrects the poor conditioning and provides a direction aligned with the valley geometry. Armijo stabilizes the method far from the minimum but allows large, efficient steps near it.
Newton + Armijo is the most stable and efficient method for this problem.

Newton's Method with Constant Step Size $\alpha = 1$ This method also converges (5 iterations), but only because the starting point is not too far from the minimizer.

Mathematical interpretation. Near the minimum, the Rosenbrock function behaves almost quadratically, and Newton's method is ideal for quadratic functions. However, without line search, Newton can diverge if:

- the Hessian is not positive definite,
- the initial point is too far from the valley,
- the Newton direction points outside the stable region.

Newton with fixed step size may converge very quickly, but it is potentially unstable.

Method	**Converges?**	**Iterations**	**Key Interpretation**
Gradient Descent + Armijo	Yes	22378	Gradient is misaligned with the narrow Rosenbrock valley; Armijo forces extremely small steps, causing very slow progress.
Gradient Descent + $\alpha = 1$	No	100000	Step size is too large; iterates oscillate and repeatedly leave the curved valley, preventing convergence.
Newton + Armijo	Yes	**15**	Hessian information corrects curvature; Armijo ensures stability even far from the minimum.
Newton + $\alpha = 1$	Yes (this case)	**5**	Extremely fast near the minimum thanks to quadratic convergence, but this fixed step can be unstable for general nonlinear functions.

Main Points

- **Gradient descent suffers under poor conditioning.** The steepest descent direction is a local direction of maximum slope, not the direction toward the global minimizer.
- **Fixed step sizes rarely work for difficult functions.** The curvature can change drastically, so using the same step everywhere is unsafe.
- **Armijo line search prevents divergence.** It guarantees decrease of the function but may significantly slow down progress if the descent direction is poor.
- **Newton's method is extremely powerful in narrow, curved valleys.** Its curvature correction provides directions well aligned with the geometry of the function.
- **But Newton needs line search far from the minimizer.** Without it, the method can diverge because the Hessian may not be positive definite.

1.3.4 Part d)

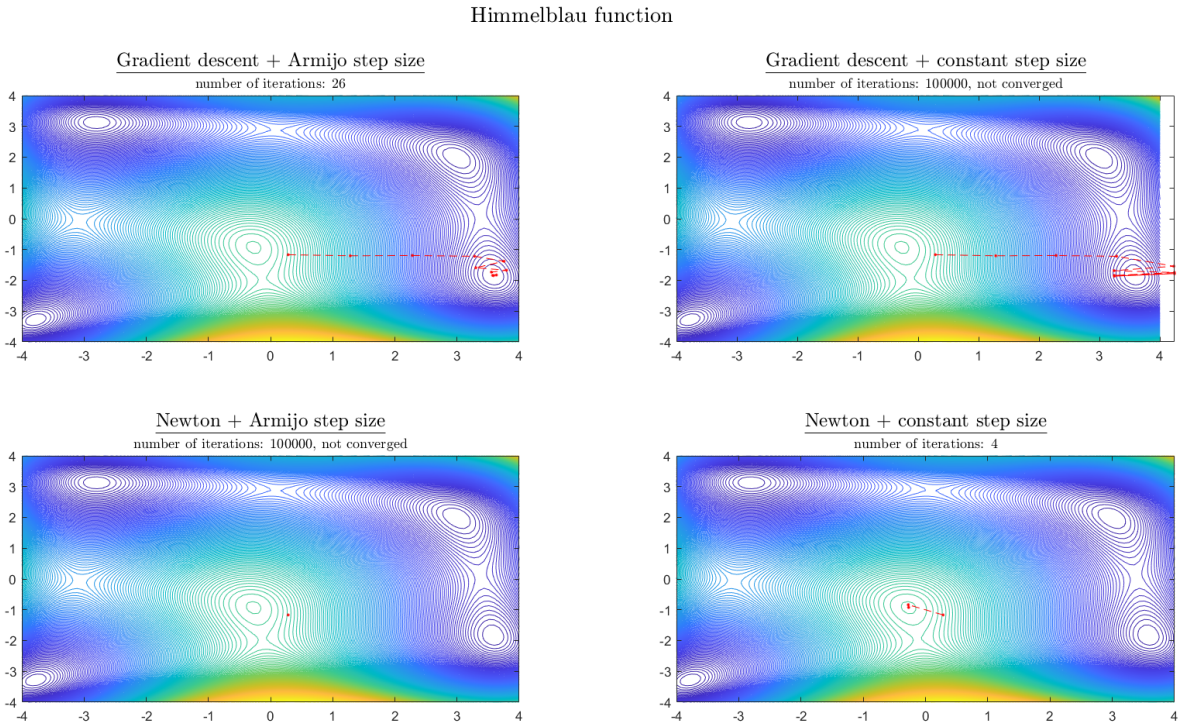


Figure 1.7: Himmelblau Comparison

Why does Newton's method with Armijo stay at x_0 for many starting points?
Newton's method computes the search direction using

$$d_k \;=\; -H(x_k)^{-1}\,\nabla f(x_k),$$

where $H(x_k)$ is the Hessian at the current iterate x_k.
For the Armijo rule to accept a step, the direction d_k must be a *descent direction*, that is,

$$\nabla f(x_k)^\top d_k < 0.$$

The Armijo condition itself is

$$f(x_k + \alpha d_k) \;\leq\; f(x_k) + \sigma\,\alpha\,\nabla f(x_k)^\top d_k.$$

However, for the Himmelblau function, the Hessian $H(x)$ has the following properties:

- it is not positive definite at many points,
- it can be *indefinite* (one positive and one negative eigenvalue),
- it can be nearly singular,
- and the function contains several minima, saddle points, and highly curved regions.

When the Hessian is not positive definite, the Newton direction

$$d_k = -H(x_k)^{-1}\nabla f(x_k)$$

is not guaranteed to be a descent direction. In fact, one frequently obtains

$$\nabla f(x_k)^\top d_k > 0,$$

which means the direction *increases* the function instead of decreasing it.
If $\nabla f(x_k)^\top d_k > 0$, the Armijo condition can *never* be satisfied, no matter how small the step size α becomes. Armijo repeatedly reduces the step size:

$$\alpha \leftarrow \beta\,\alpha,$$

but since the direction itself is not a descent direction, the inequality

$$f(x_k + \alpha d_k) \;\leq\; f(x_k) + \sigma\,\alpha\,\nabla f(x_k)^\top d_k$$

is impossible to satisfy.
Therefore, $\alpha \to 0$ and Newton's method does not move from the initial point:

$$x_{k+1} = x_k = x_0.$$

This is why Newton's method together with the Armijo step size often remains stuck at x_0 for many random starting points when minimizing Himmelblau's function.

Type of Hessian	**Eigenvalue Condition**	**2Œ2 Test**	**Intuitive / Geometric Meaning**
Positive definite	$\lambda_1 > 0,\ \lambda_2 > 0$	$\det(H) > 0,\ H_{11} > 0$	Curves upward in all directions. Looks like a bowl or cup opening upward.
Negative definite	$\lambda_1 < 0,\ \lambda_2 < 0$	$\det(H) > 0,\ H_{11} < 0$	Curves downward in all directions. Shape similar to an upside-down bowl (mountain peak).
Indefinite	One eigenvalue > 0, one < 0	$\det(H) < 0$	Some directions go up, others go down. Typical saddle shape like a horse saddle.
Semi-definite	One eigenvalue is 0 and the other ≥ 0 or ≤ 0	$\det(H) = 0$	Flat in at least one direction; curvature missing in one axis. Looks like a plate or a tube.

Table 1.1: Classification of Hessians with intuitive geometric interpretation.

Exercise sheet 5 for Numerical Optimization

Dr. Michael Lehn Tobias Born

Deadline: November 19, 2025

Exercise 1 (**Direct search methods - Nelder-Mead method**, 1+7+1+1 points)

The Nelder-Mead method is another direct search method. For $f : \mathbb{R}^n \to \mathbb{R}$ an n-dimensional polytope consisting of $n+1$ vertices is moved across the domain according to specific rules in such a way that, hopefully, it will closely enclose the local minimum at the end. Iteratively, the vertex of the polytope with the highest function value is replaced. The new vertex is determined using four operations: reflection, expansion, contraction, shrinking. Depending on the behaviour of the function, one of these operations is performed in each iteration step.

a) The detailed procedure and algorithm can be found in the lecture notes. Read this chapter and understand why each of the four operations is performed in which situation.

b) Implement the method. Therefore, fill in the gaps in `nelder_mead.m`.

c) The MATLAB programme `test_nelder_mead_rosenbrock.m` applies your implementation to Rosenbrock's function using a random initial polytope.
Execute it and reflect on the results.

d) The MATLAB programm `test_nelder_mead_himmelblau.m` does the same to Himmelblau's function.
Execute it and reflect on the results.

Exercise 2 (**Descent methods with line search**, 6+6+3+3+3+3+3 points)

After dealing with direct search methods, we will now examine a new class of approaches, *descent methods*. These require the optimization problem to satisfy more conditions, namely that f is continuously differentiable. Descent methods generate a sequence of iterates

$$x_0, x_1, x_2, \ldots \in \mathbb{R}^n \ .$$

The $(k+1)$-st iterate x_{k+1} arises from the k-th iterate x_k as

$$x_{k+1} = x_k + \alpha_k d_k \ ,$$

where

i) the search direction $d_k \in \mathbb{R}^n$ with $\|d_k\|_2 = 1$ is chosen such that f, starting from x_k, decreases in the direction d_k.

ii) The step size $\alpha_k \in \mathbb{R}_+$ specifies how far from x_k in the direction of d_k the next iterate x_{k+1} lies.

i) Search direction: A vector $d_k \in \mathbb{R}^n$ with $\|d_k\|_2 = 1$ is called *direction*. A direction d_k is called a *search direction* if the directional derivative of f at x_k in direction d_k is negative, so

$$\nabla f(x_k)^T d_k < 0 \ .$$

That is because then there exists an interval of step sizes $(0, \tilde{\alpha})$ with $\tilde{\alpha} > 0$, such that for every $\alpha_k \in (0, \tilde{\alpha})$ it is

$$f(x_{k+1}) = f(x_k + \alpha_k d_k) < f(x_k) \ ,$$

and it is worthwhile to use this direction to determine x_{k+1} and to search for a suitable step size.

a) Proof that if a direction $d_k \in \mathbb{R}^n$ fulfils $\nabla f(x_k)^T d_k < 0$, there is an interval of step sizes $(0, \tilde{\alpha})$ with $\tilde{\alpha} > 0$ such that for $\alpha_k \in (0, \tilde{\alpha})$ it is $f(x_k + \alpha_k d_k) < f(x_k)$.

For $\nabla f(x_k) \neq \mathbf{0}$, the specific direction

$$d_k = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|_2} \tag{1}$$

is called *steepest descent direction*, as

$$\nabla f(x_k)^T \left(\frac{-\nabla f(x_k)}{\|\nabla f(x_k)\|_2} \right) = \min_{\substack{d \in \mathbb{R}^n \\ \|d\|_2 = 1}} \nabla f(x_k)^T d \ . \tag{2}$$

b) Prove (2) and prove also that (1) is the unique minimizer.

Important special cases of search directions can be written in the form

$$d_k = \frac{-B_k \nabla f(x_k)}{\|B_k \nabla f(x_k)\|_2} \quad \text{with} \quad B_k \in \mathbb{R}^{n \times n} \ .$$

For example,

- $B_k = I$ yields the *gradient descent method* with $d_k = \frac{-\nabla f(x_k)}{\|\nabla f(x_k)\|_2}$, so the direction of the steepest descent (see 1).
- For $B_k = (H_f(x_k))^{-1}$ one gets Newton's method.

ii) Determination of the step size: Now let a search direction d_k be chosen. The question then is how far to go in this direction, i.e., how large α_k should be. The intuitive idea would probably be to choose the point on the line through x_k in the direction of d_k that minimizes the function value on the line, i.e., to choose α_k such that

$$f(x_k + \alpha_k d_k) = \min_{\alpha \in \mathbb{R}_+} f(x_k + \alpha d_k) \ . \tag{3}$$

Unfortunately, this is not possible in general, as (3) would just be another optimization problem we would have to solve. Of course, for special cases of functions, e.g. quadratic ones, you can carry out an exact line search.
In practice, one looks for easily verifiable conditions on the step size so that the resulting decrease in the objective function is acceptably large. At first the *Armijo condition*. For $f : \mathbb{R}^n \to \mathbb{R}$ continuously differentiable and a constant $c_1 \in (0, 1)$, a step size $\alpha \in \mathbb{R}^n$ fulfils the Armijo condition if

$$f(x_k + \alpha d_k) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^T d_k \ . \tag{4}$$

c) Using a simple one-dimensional function, think about what the Armijo condition (4) actually means and what c_1 does.

The problem with the Armijo condition is that it is satisfied by every sufficiently small step size. The *curvature condition* is thought to prevent this. It demands that

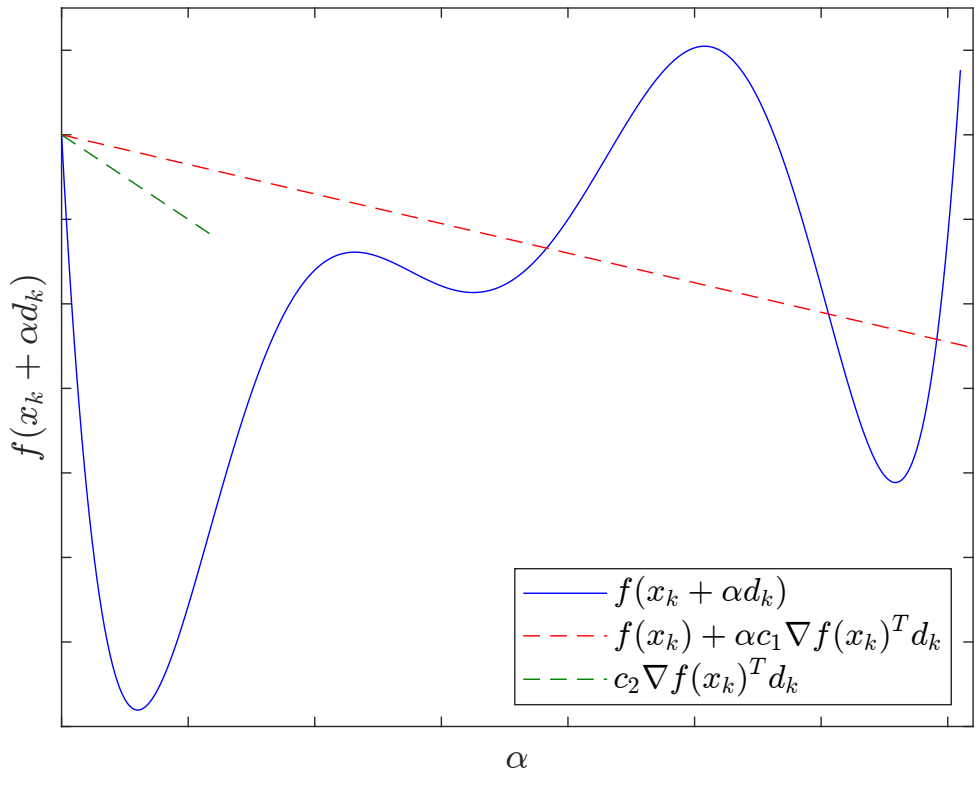
$$\nabla f(x_k + \alpha d_k)^T d_k \geq c_2 \nabla f(x_k)^T d_k \tag{5}$$

with a constant $c_2 \in (c_1, 1)$.

d) Using a simple one-dimensional function, think about what the curvature condition (5) actually means and what c_2 does.

The combination of both, the Armijo condition and the curvature condition, is known as *Wolfe conditions*.

e) Look at the function graph in the following figure. Where are which of the above conditions met?



A problem of the curvature condition is that it allows $\nabla f(x_k + \alpha d_k)^T d_k$ to have a large value. Therefore, the *strong Wolfe conditions* demand an even stronger condition on the curvature. The Armijo condition remains the same, but the curvature condition is replaced by

$$|\nabla f(x_k + \alpha d_k)^T d_k| \leq c_2 |\nabla f(x_k)^T d_k| \tag{6}$$

with some constant $c_2 \in (c_1, 1)$.

f) Using a simple one-dimensional function, think about what the condition (6) actually means.

g) Consider the figure from exercise e). Where are the strong Wolfe conditions met?

We defined the step size rules differently on the exercise sheet than in the lecture notes. Here, we defined a general condition, (4), for Armijo step sizes that can be satisfied by multiple α. In practice, the question now arises as to how an Armijo step size can be determined in a meaningful way. The aim is to get a step size that fulfils the Armijo condition, is as large as possible and at the same time it is not a lot of effort required to determine the step size. A possible approach is to set a maximum step size and reduce it until the Armijo condition is satisfied. And that is exactly how the Armijo step size is defined in the lecture notes. There, the Armijo step size is a unique step size and its definition directly specifies a procedure for its determination.

Exercise 3 (**Implementation of a general descent method**, 5+5+1+2 points)

We want to implement a general descent method so that the search direction and the step size determination can be passed as function handles.

a) Create a MATLAB program `descent_method.m` that implements a general descent method. It should receive a starting point `x0`, function handles

   - `grad_f(x)` for the function's gradient,
   - `direction(x)` for the search direction and
   - `step_size(x,d)` for the step size,

   where `x` is a point in the domain and `d` is the respective search direction, as well as a maximum number of iteration steps `max_it` and a tolerance `tol`.
   It is supposed to terminate when the norm of the function's gradient is smaller than the tolerance or the maximum number of iteration steps is reached. It is supposed to return a vector of dimension $(2 \times (\text{number iteration steps} + 1))$ containing all iterates.

We want to test two choices of step sizes, in particular the Armijo step size.

b) Create a MATLAB program `armijo_step_size.m` that determines the unique Armijo step size as defined in the lecture notes. Actually, its definition directly gives the procedure to compute it. The program is supposed to receive the current iterate `x`, the search direction `d`, a function handle `f`, a function handle `grad_f` and the two parameters `beta` and `sigma` as defined in the lecture notes. It should return the Armijo step size.

We want to test two different search directions, namely the *gradient descent method* and *Newton's method*, on the examples of Rosenbrock's function and Himmelblau's function. We want to try as well the Armijo step size as a constant step size of one.

c) The MATLAB program `test_descent_methods_rosenbrock.m` performs your descent method with all four combinations of search directions and step sizes.
   Fill the gaps in the function handles for the search directions. For the gradient descent, normalize the length of the search direction to one before determining the step size. For Newton's method, do not do that.
   Then execute the program and reflect on the outcome.

d) Do the same with the MATLAB program `test_descent_methods_himmelblau.m`, which considers Himmelblau's function.
   Execute it and reflect on the outcome.
   Newton's method in combination with the Armijo step size stays at `x0` for many random starting points. Why is this the case?